

ECE391

Computer System Engineering

Lecture 1

Prof. Kirill Levchenko and Dong Kai Wang

Slides adapted Prof. Kalbarczyk and Lumetta

University of Illinois at Urbana- Champaign

Spring 2024

Course Introduction

- Course philosophy
- Course logistics
- Advice from alumni

Who are We?

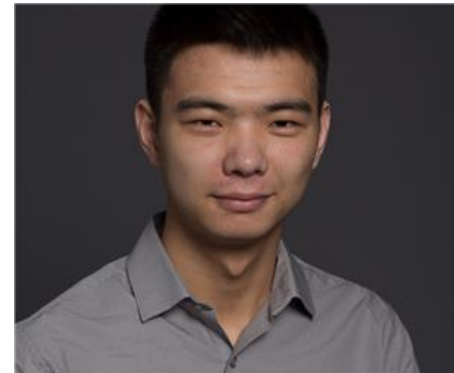


**Kirill
Levchenko**

Associate Prof. ECE, CSL
CSL 458

Research: cyber-physical system security, network security, e-crime, and internet service abuse.

klevchen@illinois.edu



**Dong Kai
Wang**

Tch. Asst. Prof. ECE
ECEB 2066

Research: computer architecture, hardware acceleration, models of computing, and ML for systems.

dwang47@illinois.edu

And Your New Best Friends...

Linghao Zhang

Eli Feinberg

Sanjeevi Sengottuvel

Yueze (Hyouin) Liu

Zhongbo Zhu

Yue Yuan

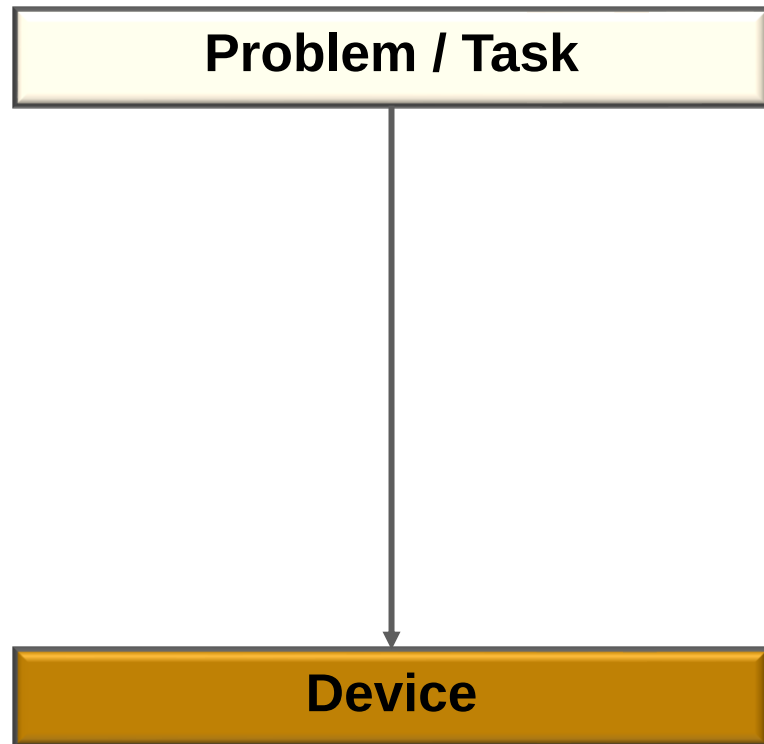
Deniz Caglar

and many CAs!

All of these people **have done well in 391**
and they can help you!

<https://courses.grainger.illinois.edu/ECE391/sp2024/staff.html>

Review: Layers of Abstraction



What's $\$179.61 + \211.39 ?

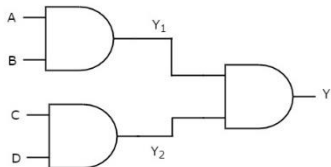


Review: Layers of Abstraction



>"hello
world!"

AND	0101	DR	SR1	1	imm5
NOT	1001	DR	SR		111111
BR	0000	n	z	p	PCoffset9



Problems / Tasks

Algorithms

Application Software

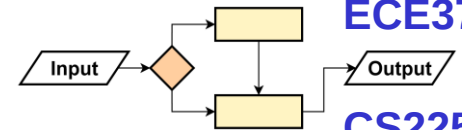
OS / Systems Software

Architecture / ISA

Microarchitecture

Digital / Analog Circuits

Devices



ECE374

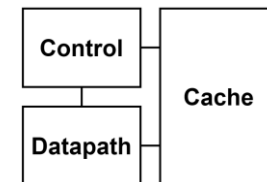
CS225

ECE220



ECE391

ECE411



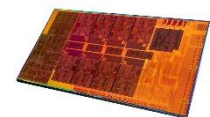
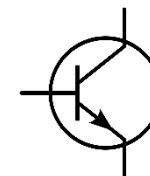
ECE512

ECE385

ECE425

ECE482

ECE483



Why Operating Systems?

- Why don't we simply run application software directly on the CPU?
 - Like you did on the LC3 for ECE120, 220, 385.
- Also, it is known that OS has nontrivial performance impacts (*addr. translations, cache misses, etc.*).

Characterizing the Caching and Synchronization Performance of a Multiprocessor Operating System

Josep Torrellas, Anoop Gupta, and John Hennessy
Computer Systems Laboratory
Stanford University, CA 94305

<https://dl.acm.org/doi/10.1145/143371.143506>

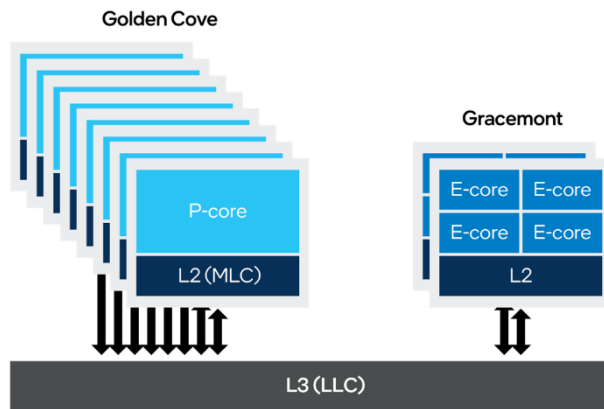
- **So why do we need the OS?**

Why Operating Systems?

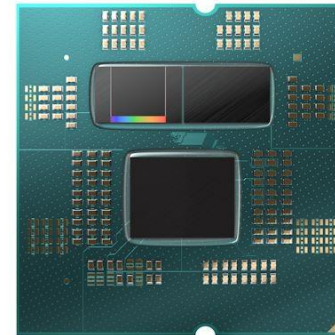
- **Abstraction**
- **Virtualization**
- **Concurrency / Multitasking**
- **Resource Management**
- **Persistence**
- **Security**
- **User Interface**
- ...

A More Recent Motivating Example

- Hardware is constantly changing.
 - Recent examples: Intel Alder Lake, AMD 3D V-Cache, ...



CPU has different types of cores



CPU has different types of caches

- *Why is the OS important here?*

Linux Snatches Alder Lake Productivity Crown From Windows 11

News

By Mark Tyson published February 14, 2022

Intel's hybrid chips are now faster in Linux after kernel 5.16+.

Ans:
Scheduling!

Knowledge Objectives

1. understand the hardware-software interface in detail.
2. understand how one abstracts physical resources in software.

Understand three key roles for operating system (OS):

- **virtualization** – providing the illusion of larger or unlimited physical resources.
- **abstraction** – providing simpler, more intuitive interfaces to hardware.
- **protection** – ensuring that distinct programs and users do not interfere with one another's operation.

Operating Systems are Key to System Software

- **Why study operating systems (OSs)?**
- OSs are the **epitome of all software**: any idea tried in software has been tried in an OS.
- **Increased abstraction**, even in embedded systems...
 - ECE391 created before phones had OSs.
 - Today, your watch may have an OS.
 - Tomorrow, perhaps an OS will run in your shoe?

Skill Objectives



1. Tools—learn to use them! They make you happy

(source control, compiler, dependency management, debugger)

2. Learn team-based development and testing

—learn to communicate, leverage individual strengths, design workable interfaces for development and test

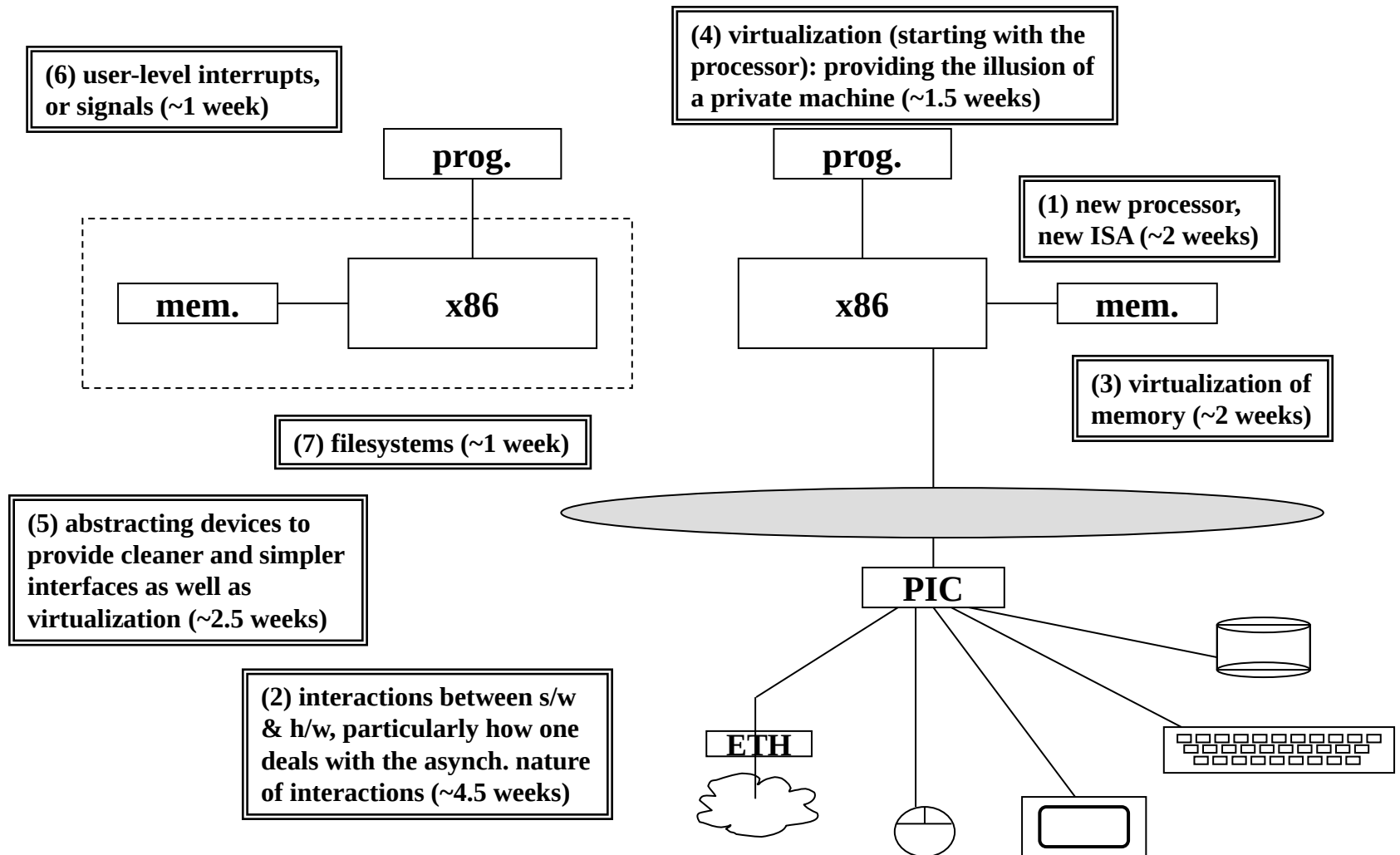
—learn to create, implement, and execute testing plans

3. Learn how to find necessary information

—reading specs and documentation

—reading code

Big Picture of the Material



- ECE 391 provided the foundation necessary to be succeed in my CPU design verification internship at Apple. Even though I wrote ARM assembly, the **fundamental ideas taught in lecture and the hours reading dense technical documentation (Intel Manual) made it easy to learn quickly** from the corresponding ARM manuals and Apple documents.
- Eric Clark, ECE 391 in Fall of '13, now at Apple

Coding in Practice

- However, the value of the ECE 391 to me extends beyond memorizing particular concepts; it was in ECE 391 that I got my first real experience working in unfamiliar codebases and debugging challenging problems. **In my professional career, I have found that much of my time is spent reading new code and figuring out how to implement new features that that my org needs;** these skills have transferred well from ECE 391.
- Matt Tischer, Spring '12, Software Engineer at Microsoft

Real Systems are Messy

- ECE 391 is pretty much the only class in college that challenged my coding skills. ECE 190 ... and CS 225 gave me some exposure to coding, but at that point I was still learning. 391 was the first time that I got to actually work on a large code. **In my job today, I don't have the luxury of the answers already being figured out... There are no instructions, no hints, no TA office hours, no nothing.** It's just me and hours of banging my head against a wall. But I'd like to think that 391 prepared me well and so I suffer from much less head-banging than I likely would have otherwise.
- Eric Carl of House Badger, the First of His Name, The Unallocated, King of the Stackman, the Bitdiddle and the First Ben, King of Execute, Kerneleesi of the Great Language C, Protector of the Stack, Lord Regnant of the Seven Syscalls, Breaker of Segments and Father of Pages, Spring 2012, Hadoop Code Monkey at Oath: the company formerly known as Yahoo!

Writing Good Code

- There were so many things I learnt in ECE 391, but the most important one is **how to write good code and why do we need to do so**. ECE 391 prepared me really well for my job, aside from the point mentioned above, it also taught me how to think as an computer system engineer.
- Fei Deng, Spring'14, Software Development Engineer at Yahoo! Inc.

Systems Thinking

- Computer systems and cars are similar machines: in the sense that you don't have to know how to build a car in order to drive or fix one, nor will it make you an expert driver or technician (software engineering and I.T. in the computer world), but **if you understand the fundamental physics you will never be surprised with something you can't understand or learn.**
- Daniel Fernandes, Fall of '13, EE PhD Student at Stanford University

Systems Thinking II

- 391 benefited me greatly in my career ... Even with architectural differences of working on Windows, **391 helped teach me how to approach and solve system-level problems.** I also spend a significant amount of time debugging crash dumps, so the debugging experience from the MPs was very helpful, as well as being able to look at disassembly with the knowledge of x86 and calling conventions.
- Drew Kluemke, Fall 2015, Microsoft

Debugging

- I think taking ECE 391 has earned me an enormous amount of respect in the workplace ... one other thing I think is hugely important in ECE 391 is that you learn to be very methodical in how you debug and write code ... I think this skill, perhaps more than any other, will impress your coworkers and bring value. **I've had cases where people asked me "how did you find that bug?" when the answer was simply I opened the debugger and stepped through the code.**
- Dennis Liang, Spring '14, Systems Engineer at Cloudflare

Groups

- ECE 391 gave me experience in working on a large **coding project within a group**... The course also taught me how to better **collaborate with group members** and divide work in ways so group/team members don't "step on each other's toes."
- Michael F., Fall 2015, now Software Engineer at Microsoft.

Course Elements

- Lectures
 - Focus is on concepts and case studies.
- Discussion sections
 - Help with lecture concepts and labs.
- Pre-labs
 - Prepare you for the labs.
- Labs/Machine Problems
 - Experience with real systems.
- Exams (2 midterms + 1 final)
 - Cover **both** lecture and lab material.
- Class Website:
 - <https://courses.grainger.illinois.edu/ece391/sp2024/>

Course Materials: Notes and References

- Course notes terse and to the point (online).
- Books—more detail than needed, but good as references and introductory material:
 - *Understanding the Linux Kernel* (Bovet & Cesati, 3rd ed)
 - *Linux Device Drivers* (Corbet et al., 3rd ed)
 - *Linux Kernel Development* (Love, 3rd ed)
 - *Advanced Unix Programming* (Rochkind, 2nd ed)
- See web page for more resources:
<https://courses.grainger.illinois.edu/ECE391/sp2024/references.html>

Information about the Class

- Piazza
 - Please join if you haven't (<https://piazza.com/illinois/spring2024/ece391/>).
 - Stop here first!
 - All announcements & clarifications posted here.
 - Can ask public and private questions.
 - Limited anonymity.
 - **DO NOT POST SOLUTIONS.**
 - **Don't repeat questions (search before asking).**
- Canvas (Grades)
- Office Hours
 - Schedule can be found on course site.
- Email
 - Last resort!
 - Must include "[ECE391]" in subject!

Workload and Grading

- Workload includes
 - four programming assignments, two with pre-lab,
 - two midterms, and one final exam.
- Grading:
 - **50% on labs** (5%, 10%, 10%, 25%)
(prelabs worth 10% of associated lab grade),
 - **15%** on each exam, and
 - 5% subjective evaluation (mainly, peer-evaluations).

What About Discussion Section?

No points for discussion section?

- Technically, no.
- But they were **created in response to student demand.**
- They **will help you do better** in the course.
- We **STRONGLY ENCOURAGE** you to attend.
- Go to one, go to two, whatever you like.**

** We do have to obey fire marshal rules about people in any space.

Exam Timing

When are the exams?

- Exam 1: Tuesday February 20, 7:00-9:00 PM.
- Exam 2: Monday April 8**, 7:00-9:00 PM.
- Final: TBD

*** Tentative date, may change.*

Collaboration Allowed for Prelabs

- OK to talk to staff at any time.
- **Pre-labs are completely open**—work with anyone
 - We encourage you to do so.
 - **Don't screw yourself** by putting your name on something you didn't do.
 - **Turn in one copy with all names and NetIDs** to be credited (save us time on grading).
 - (if your name is on >1 submission, you earn the minimum grade over all those with your name).

Collaboration NOT Allowed for Labs and Exams

- Machine problems (labs):
 - **MPs 0, 1, and 2 are individual work**
 - **MP3 is done with a team of four**
 - **collaboration is NOT allowed**
 - OK to talk with others about ideas, strategies.
- Exams — all by yourself.

Academic Integrity Violations...

Just Don't Do It

- **Any unauthorized provision or use** of code
 - not written by you (or your partners, for MP3).
 - **is a violation of academic integrity.**
- Such violations will result, at a minimum, of
 - **0 on the assignment,**
 - **loss of one full letter grade,** and
 - a report to the college through FAIR (Faculty Academic Integrity Report).
- **Additional violations will result in failing the class.**

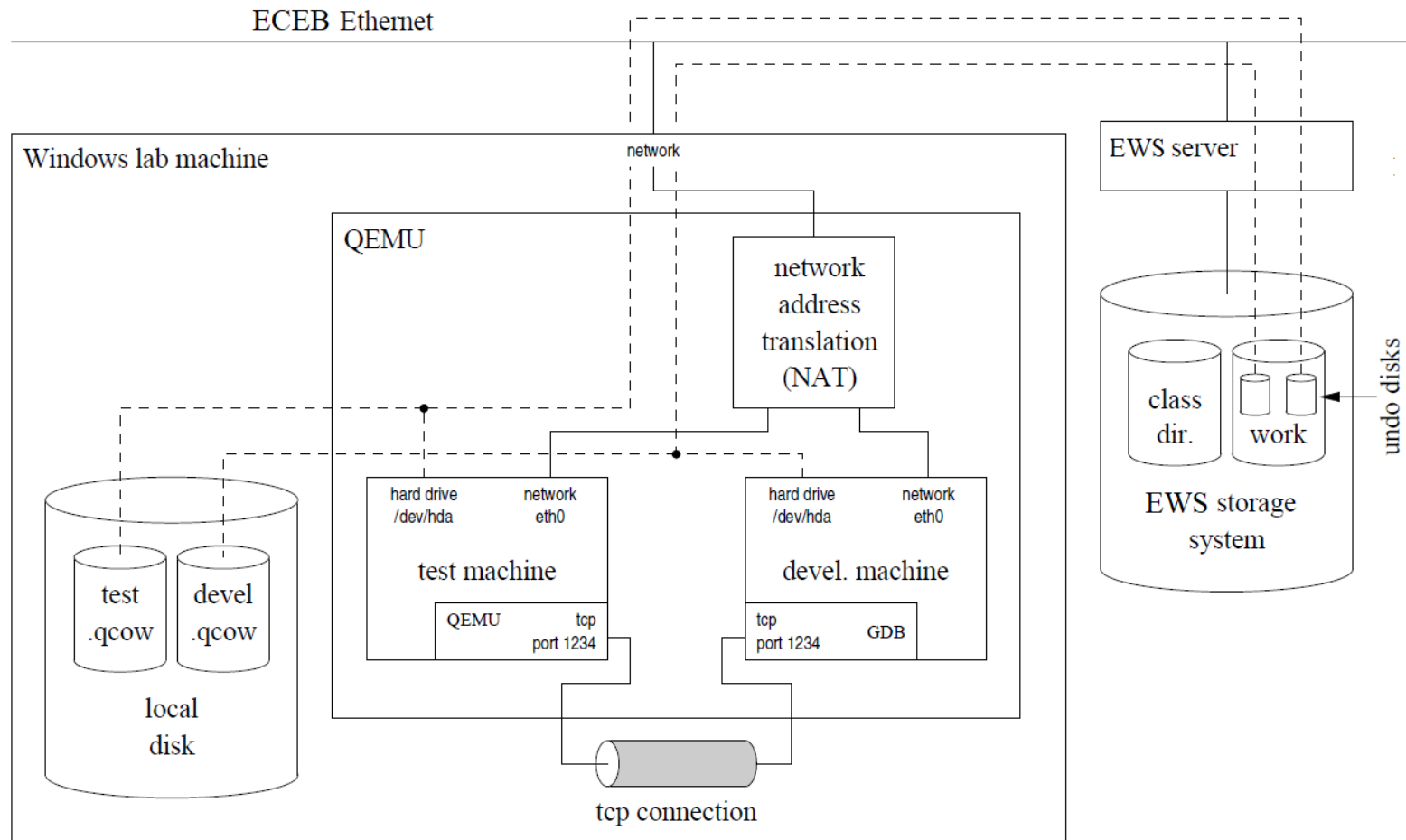
Examples of Prohibited Actions

- **Turning in any portion of another's code as your own** (whether from a current student, a former student, a Github repo, a Co-Pilot session, a Git-a-Coder payment, or anything else).
- **Making your solution code available to a current student** (whether by printing it, posting it to Github, sending by email, or any other means).
- **Using another's code** (see list above) **as a hint or template** to develop your own solution.

MP3 Offers a Design Competition

- In **MP3**, in the **second half of the semester**,
 - you'll work with a **team of four** (not 3, not 5)
 - **to develop your own OS.**
- For the hyper-motivated,
 - we have a **design competition**,
 - **winner gets a prize.**

ECE391 Lab Environment



MP0 ... Have You Started Yet?

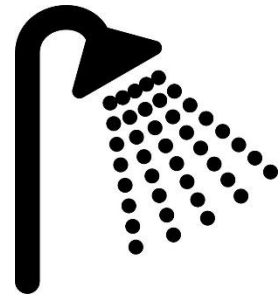
- Set up your personal environment in **MP0**,
 - **due** in (any TA/CA's) office hours **by Next week**
 - **Demo in person in 3026 ECEB.**
- You may also want to work on your own machines:
 - https://courses.grainger.illinois.edu/ece391/sp2024/secure/references/doc-workhome_novpn.pdf
 - allows you to develop and test MP code,
 - without competing for lab resources.
- (New for 2024!) Docker set up:
 - <https://courses.grainger.illinois.edu/ECE391/sp2024/secure/references/ece391-docker-wfh.pdf>

Lab Guidelines for MP0 and Beyond

- There will be contention for lab resources, so **start early!**
- **For MP0,**
 - you may **lock** one lab computer for **up to 4 hours**,
 - if you **leave a note** with expected return time.
- **After MP0, the lock limit is 30 minutes.**
- **Hand-in/demo takes precedence** for machine use and TA/CA time.
- Working from home is encouraged, but **demos must happen on the lab machines.**
 - **Please be respectful to others**

ECEB 3026 Lab Policies

- EWS lab policies: [EWS Policies \(uillinois.edu\)](http://uillinois.edu)
- No food or lidless drinks. A red circle with a diagonal slash over a cartoon illustration of a burger, fries, and a drink.
- No sleeping in labs.
- Headphones shouldn't be heard by others!
- Do not use our lab for other classes or purposes.
- Take care of personal hygiene!
 - I will be patrolling!
- Log out and clean up after you leave.
- In summary, be respectful to equipment and others...





*MORE ADVICE FROM THE
AGES*

Right Attitude

- I think ECE 391 can be a bit of a grind for some students because of its difficulty. However, consider this: **it's *worth* learning because it's difficult**. You shouldn't shy away from learning difficult things, you should welcome it. Learning doesn't stop when you're done with ECE 391, ...
- It is an important class, and a difficult one for many students, but **you don't have to be a genius to do well**. The same hard work and dedication that got you here will carry you through this class, too.

- My advice is, and has always been, **RTDC (Read the Documentation Carefully)**. Read whatever the class provides at least twice before starting to solve the problem or write code. There were so many times students ask questions that can be answered with a direct quote from the docs we gave out.

The TAs Don't Suck

- Go to discussions, pay attention to the TA since they've all done well in ECE 391. **Go to office hours, ask good questions but don't expect the TA to help you debug.**
- Talk to the TAs and your classmates. **Make sure you really understand what the class is asking you to do and why** you're doing it.
- **Make use of office hours.** The 391 staff is relatively large and hosts frequent office hours in the lab. Maximize the value of the questions you ask; "It looks like I'm clobbering my stack somewhere in lines 10-20" is more likely to get useful results than "something is wrong with my code".

- **Get comfortable with your debugging setup, you'll be using it a lot.**
 - Learn to use GDB, don't rely on "printf" to debug, that's what newbies do.
 - Check the GDB reference materials on course website.
- Breaking down large tasks into smaller segments of code and **testing** each segment **early and often** really makes debugging easier as there are less items that can go wrong at each step.

Time Management

- **Starting early** is the most obvious but also most helpful piece of advice. If you spend a few hours a day on 391, you'll be a lot more effective than trying to complete a checkpoint within a 48-hour timespan. As a TA, I saw so many groups that were trying to rush to complete checkpoints in a day or two, and all of them looked miserable.
- My advice to ECE 391 students is to **start early** on assignments and stay ahead of deadlines. Bugs will crop up at the worst possible moment.
- Everything will **take longer than you think**, so you need to plan ahead if you want to sleep around finals week.
- **Time management in ECE 391 is critical.** While it may be possible to complete MPs in a single night in some other classes, ECE 391 is not among them.
- **Plan ahead.** This sounds so simple, but it's actually the hardest thing to do, because of the temptation of cutting corners. It may feel like a waste of time, but it will save you countless hours of debugging in the future.